Informe tipo · Reto «Herramientas internas»

0 · Cómo usar este informe

- Es un **informe de ejemplo**, con los resultados reales de la solución de referencia.
- Las tablas tienen una columna **«Vuestro»** en blanco: rellenadla con lo que sale en vuestro ordenador.
- Si una cifra no coincide, ya sabéis **dónde mirar**: cada cifra indica de qué función sale.
- Sirve de guion para el README: qué se hizo, qué salió y qué decisiones tomasteis.

1 · El reto en una página

El encargo

- **Cliente:** TurisData.
- **Problema:** hoy calculan sus indicadores a mano, en una hoja de cálculo.
- **Entregable:** una herramienta, `kpis.py`, que con **un solo comando** lee el CSV de reservas y calcula tres indicadores.

El comando

```
python kpis.py reservas_turisdata.csv salida.json --capacidad 40
```

El resultado de un vistazo

Qué	Resultado	Vuestro
Reservas leídas del CSV	6000	
Filas descartadas	0	
Indicadores calculados	3 (cancelación por canal, ingreso medio, ocupación aproximada)	
Fichero generado	`salida.json`	
Errores provocados y controlados	3 de 3	
Pruebas superadas	**12 de 12**	

2 · Qué hace el programa

El recorrido de los datos

```
reservas_turisdata.csv
        │
        ▼
leer_reservas ──(usa)──► _convertir_fila      texto → números y fechas; descarta filas malas
        │  devuelve: reservas buenas + avisos
        ▼
calcular_kpis ──(usa)──► tasa_cancelacion_por_canal
        │                ► ingreso_medio_por_reserva
        │                ► ocupacion_proxy
        ▼
exportar_json                                  guarda el resultado
        │
        ▼
salida.json            + mensaje «OK» y código de salida 0
```

Quién manda: `main`

- Recibe los argumentos de la terminal: CSV de entrada, JSON de salida y capacidad.
- Llama a las funciones **en orden**.
- Si falta el fichero o una columna: imprime `ERROR: …` por `sys.stderr` y termina con **código 1**.
- Si todo va bien: imprime `OK: …` y termina con **código 0**.

3 · Resultados con el CSV real

3.1 · Los datos leídos

Cifra	Resultado	Sale de	Vuestro
Reservas válidas	6000	`leer_reservas`	
Filas descartadas	0	`leer_reservas`	
Periodo de llegadas	del 13/01/2024 al 04/12/2026 (1057 días)	`ocupacion_proxy`	
Reservas confirmadas	3246	`cancelada == 0`	
Reservas canceladas	2754 (45,9 %)	`cancelada == 1`	

3.2 · Indicador 1 · Tasa de cancelación por canal

Cómo se calcula: canceladas del canal ÷ reservas del canal × 100, con 1 decimal. Función: `tasa_cancelacion_por_canal`.

Canal	Reservas	Canceladas	Tasa	Vuestra
`web_intermediaria`	3020	1628	**53.9 %**	
`agencia`	1190	524	**44.0 %**	
`empresa`	328	142	**43.3 %**	
`directo`	1462	460	**31.5 %**	

3.3 · Indicador 2 · Ingreso medio por reserva

Cómo se calcula: (noches × precio por noche) de cada reserva **no cancelada**, sumado y dividido entre cuántas son. Función: `ingreso_medio_por_reserva`.

- Reservas confirmadas: **3246**.
- Ingreso total de las confirmadas: **1.869.826,34 €**.
- Ingreso medio por reserva: **576.04 €**. Vuestro: _____
- Las **canceladas no cuentan**: no dejan dinero.

3.4 · Indicador 3 · Ocupación aproximada («proxy»)

Cómo se calcula: noches de reservas no canceladas ÷ (días del periodo × habitaciones) × 100. Función: `ocupacion_proxy`.

- Noches confirmadas: **17 756**.
- Días del periodo: **1057** (se cuenta también el primer día).
- Habitaciones: **40** (`--capacidad 40`).
- Cuenta: 100 × 17 756 ÷ (1057 × 40) = **42.0 %**. Vuestro: _____

Qué pasa si cambia la capacidad

Capacidad (`--capacidad`)	Ocupación	Vuestra
20 habitaciones	84.0 %	
40 habitaciones	42.0 %	
80 habitaciones	21.0 %	

- Al **doblar** la capacidad, la ocupación se queda en la **mitad**.
- Si no es así, `args.capacidad` no está llegando a `ocupacion_proxy`.

3.5 · El fichero `salida.json`

```
{
  "reservas_analizadas": 6000,
  "ocupacion_proxy_pct": 42.0,
  "ingreso_medio_por_reserva": 576.04,
  "tasa_cancelacion_por_canal_pct": {
    "empresa": 43.3,
    "directo": 31.5,
    "web_intermediaria": 53.9,
    "agencia": 44.0
  },
  "calidad": {
    "filas_validas": 6000,
    "filas_descartadas": 0,
    "avisos": []
  }
}
```

- Los tres primeros bloques son los **indicadores**.
- `calidad` dice **cuánto se puede fiar uno del resultado**: cuántas filas eran buenas, cuántas se descartaron y por qué.

3.6 · Qué dicen las cifras (y qué no)

Lo que dicen

- Casi **la mitad de las reservas (45,9 %) se cancelan**.
- **`web_intermediaria`** es el canal con más reservas (3020 de 6000, el 50,3 %) y el de mayor tasa de cancelación (53.9 %).
- **`directo`** es el canal con menos cancelaciones (31.5 %).
- Cada reserva confirmada deja de media **576.04 €**.

Lo que no dicen

- **No explican por qué** se cancela más en un canal: eso necesitaría más análisis.
- La ocupación es un **proxy**: el fichero no trae la capacidad real del hotel, y cada reserva cuenta todas sus noches el día de la llegada. Sirve para **comparar**, no es una ocupación exacta.

4 · Los errores, a propósito

Se provocan **sobre copias** del CSV, para comprobar que el programa **no se rompe** (nunca sale un `Traceback`).

Caso	Comando	Sale	Código	Vuestro
Fichero que no existe	`python kpis.py no_existe.csv salida.json`	`ERROR: No se encuentra el fichero: no_existe.csv`	`1`	
Falta una columna	`python kpis.py sin_col.csv salida_prueba.json`	`ERROR: Faltan columnas obligatorias: ['precio_noche']`	`1`	
Dos filas con datos incorrectos	`python kpis.py con_error.csv salida_prueba.json`	`OK: 5998 reservas analizadas, 2 descartadas -> salida_prueba.json`	`0`	

Qué demuestra cada caso

- **Fichero o columna que faltan:** no hay nada que calcular, así que el programa **para** con un mensaje claro y código `1`.
- **Filas malas:** el programa **sigue**, descarta esas filas y las apunta en `calidad.avisos` (`Fila 5 descartada: …` y `Fila 10 descartada: noches debe ser >= 1`).
- Descartar una fila **sin avisar** sería peor que pararse: nadie sabría que faltan datos.

5 · Las 12 pruebas

Prueba	Qué comprueba	Función	Misión
`test_tasa_cancelacion_por_canal`	Las tasas con 5 reservas, y `{}` sin reservas	`tasa_cancelacion_por_canal`	3
`test_tasa_cancelacion_se_redondea_a_un_decimal`	1 de 3 = 33.3 y 2 de 3 = 66.7	`tasa_cancelacion_por_canal`	3
`test_ingreso_medio_por_reserva`	La media de las confirmadas, y 0.0 sin reservas	`ingreso_medio_por_reserva`	3
`test_ocupacion_proxy`	14.0 %, 0.0 sin reservas y error con capacidad 0	`ocupacion_proxy`	3
`test_leer_reservas_convierte_tipos`	Tipos buenos: `int`, `float` y fecha	`leer_reservas` y `_convertir_fila`	4
`test_fichero_inexistente`	Error claro, con el nombre del fichero	`leer_reservas`	4
`test_faltan_columnas`	Error que nombra la columna que falta	`leer_reservas`	4
`test_filas_defectuosas_se_descartan_y_se_avisan`	4 filas malas → 4 avisos	`leer_reservas`	4
`test_valores_imposibles_se_descartan`	0 noches, precio 0, `cancelada` 2… no valen	`_convertir_fila`	4
`test_datos_reales_si_estan_disponibles`	Los 6000 datos reales y sus cifras	todas las anteriores	3 y 4
`test_exportar_json_y_main`	El programa entero, de principio a fin	`exportar_json` y `main`	5
`test_version_pandas_coincide_si_existe`	Opcional: la versión con pandas da lo mismo	`kpis_pandas.py`	extra

6 · Cómo se ha construido kpis.py

6.1 · De la plantilla al entregable

El marcador de `python test_kpis.py`, etapa por etapa

Etapa	Qué se añade a `kpis.py`	Pruebas superadas	Vuestro
Plantilla	Nada: cada función tiene una línea `raise NotImplementedError`	1 de 12	
Misión 3	`tasa_cancelacion_por_canal`, `ingreso_medio_por_reserva`, `ocupacion_proxy`	5 de 12	
Misión 4	`_convertir_fila`, `leer_reservas`	11 de 12	
Misión 5	`exportar_json`, `main`	**12 de 12**	

- La **1 de 12** de la plantilla es la prueba opcional de pandas, que se omite y cuenta como superada.
- Las **misiones 0, 1 y 2** no añaden nada a `kpis.py`: son los ejercicios que preparan lo necesario (diccionarios y listas, `if`, `for` y `.get`).

6.2 · El mapa del fichero

Orden	Función	Misión	Líneas escritas	Qué recibe	Qué devuelve	Usa
1	`tasa_cancelacion_por_canal`	3	11	lista de reservas	diccionario `{canal: %}`	—
2	`ingreso_medio_por_reserva`	3	7	lista de reservas	un número (€)	—
3	`ocupacion_proxy`	3	13	reservas y habitaciones	un número (%)	—
—	`calcular_kpis`	**ya viene hecha**	5	reservas y habitaciones	diccionario con los 3 KPI	las tres anteriores
4	`_convertir_fila`	4	15	una fila de **texto**	un diccionario con tipos buenos	—
5	`leer_reservas`	4	19	la ruta del CSV	`(reservas, avisos)`	`_convertir_fila`
6	`exportar_json`	5	2	datos y una ruta	nada: escribe el fichero	—
7	`main`	5	16	los argumentos de la terminal	código de salida 0 o 1	`leer_reservas`, `calcular_kpis`, `exportar_json`

- En total, unas **83 líneas escritas por vosotros** (más 5 de `calcular_kpis`, que ya venían hechas).
- Las funciones de la **misión 3** trabajan con listas en memoria, **sin ficheros**: se pueden probar solas.
- `_convertir_fila` tampoco necesita ficheros: recibe una sola fila.
- `main` va **la última** porque necesita todas las demás.

6.3 · Qué se aprende en cada función

- **`tasa_cancelacion_por_canal`:** diccionarios, `.get(canal, 0)`, `for`, `if`, `round`.
- **`ingreso_medio_por_reserva`:** listas, `append`, `sum` y `len`, y evitar dividir entre cero.
- **`ocupacion_proxy`:** fechas, `max` y `min`, y lanzar un error a propósito (`raise ValueError`).
- **`_convertir_fila`:** convertir tipos (`int`, `float`, `date.fromisoformat`) y **validar** valores.
- **`leer_reservas`:** abrir ficheros (`with open`), `csv.DictReader`, `try` / `except`, devolver **dos cosas** a la vez.
- **`exportar_json`:** escribir JSON (`json.dump`, `ensure_ascii=False`).
- **`main`:** encadenar todo, `sys.stderr` y códigos de salida.

6.4 · El `kpis.py` final, por bloques

Bloque 0 · lo que ya viene en la plantilla

```
import argparse
import csv
import json
import sys
from datetime import date

COLUMNAS_OBLIGATORIAS = ["fecha_llegada", "noches", "precio_noche", "canal", "cancelada"]
```

Bloque 1 · Misión 3 · las tres funciones que calculan

```
def tasa_cancelacion_por_canal(reservas):
    """Devuelve {canal: porcentaje de reservas canceladas, con 1 decimal}."""
    total = {}
    canceladas = {}
    for r in reservas:
        canal = r["canal"]
        total[canal] = total.get(canal, 0) + 1
        if r["cancelada"] == 1:
            canceladas[canal] = canceladas.get(canal, 0) + 1
    tasas = {}
    for canal in total:
        tasas[canal] = round(100 * canceladas.get(canal, 0) / total[canal], 1)
    return tasas


def ingreso_medio_por_reserva(reservas):
    """Ingreso medio de las reservas NO canceladas, con 2 decimales."""
    ingresos = []
    for r in reservas:
        if r["cancelada"] == 0:
            ingresos.append(r["noches"] * r["precio_noche"])
    if not ingresos:
        return 0.0
    return round(sum(ingresos) / len(ingresos), 2)


def ocupacion_proxy(reservas, capacidad_habitaciones):
    """Ocupación aproximada en %, con 1 decimal."""
    if capacidad_habitaciones <= 0:
        raise ValueError("La capacidad debe ser mayor que 0")
    if not reservas:
        return 0.0
    fechas = []
    for r in reservas:
        fechas.append(r["fecha_llegada"])
    dias = (max(fechas) - min(fechas)).days + 1
    noches = 0
    for r in reservas:
        if r["cancelada"] == 0:
            noches = noches + r["noches"]
    return round(100 * noches / (dias * capacidad_habitaciones), 1)
```

Bloque 2 · lo que reúne los tres KPI (ya viene hecho)

```
def calcular_kpis(reservas, capacidad_habitaciones):
    return {
        "reservas_analizadas": len(reservas),
        "ocupacion_proxy_pct": ocupacion_proxy(reservas, capacidad_habitaciones),
        "ingreso_medio_por_reserva": ingreso_medio_por_reserva(reservas),
        "tasa_cancelacion_por_canal_pct": tasa_cancelacion_por_canal(reservas),
    }
```

Bloque 3 · Misión 4 · leer el CSV

```
def _convertir_fila(fila):
    """Convierte una fila de textos a tipos Python. Lanza ValueError si no es válida."""
    noches = int(fila["noches"])
    precio = float(fila["precio_noche"])
    cancelada = int(fila["cancelada"])
    fecha = date.fromisoformat(fila["fecha_llegada"])
    canal = fila["canal"].strip()

    if canal == "":
        raise ValueError("canal vacío")
    if noches < 1:
        raise ValueError("noches debe ser >= 1")
    if precio <= 0:
        raise ValueError("precio_noche debe ser > 0")
    if cancelada not in (0, 1):
        raise ValueError("cancelada debe ser 0 o 1")

    return {"fecha_llegada": fecha, "noches": noches, "precio_noche": precio,
            "canal": canal, "cancelada": cancelada}


def leer_reservas(ruta):
    """Lee el CSV y devuelve (reservas, avisos)."""
    try:
        with open(ruta, newline="", encoding="utf8") as f:
            lector = csv.DictReader(f)

            faltan = []                                          # 5.4
            for columna in COLUMNAS_OBLIGATORIAS:
                if columna not in lector.fieldnames:
                    faltan.append(columna)
            if faltan:
                raise ValueError(f"Faltan columnas obligatorias: {faltan}")

            reservas = []                                        # 5.5
            avisos = []

            for n, fila in enumerate(lector, start=2):           # 5.6
                try:
                    reservas.append(_convertir_fila(fila))
                except ValueError as error:
                    avisos.append(f"Fila {n} descartada: {error}")

            return reservas, avisos                              # 5.7
    except FileNotFoundError:
        raise FileNotFoundError(f"No se encuentra el fichero: {ruta}")   # 5.8
```

Bloque 4 · Misión 5 · guardar y poner todo en marcha

```
def exportar_json(datos, ruta):
    """Guarda `datos` en un fichero JSON legible."""
    with open(ruta, "w", encoding="utf8") as f:
        json.dump(datos, f, ensure_ascii=False, indent=2)


def main(argumentos=None):
    """Punto de entrada: devuelve 0 si todo va bien y 1 si hay un error."""
    parser = argparse.ArgumentParser(description="KPIs de reservas de TurisData")
    parser.add_argument("entrada", help="CSV de reservas")
    parser.add_argument("salida", help="JSON de salida")
    parser.add_argument("--capacidad", type=int, default=40, help="habitaciones disponibles (proxy de ocupación)")
    args = parser.parse_args(argumentos)
    try:
        reservas, avisos = leer_reservas(args.entrada)
        kpis = calcular_kpis(reservas, args.capacidad)
        kpis["calidad"] = {"filas_validas": len(reservas), "filas_descartadas": len(avisos),
                           "avisos": avisos[:20]}
        exportar_json(kpis, args.salida)
    except (FileNotFoundError, ValueError) as error:
        print(f"ERROR: {error}", file=sys.stderr)
        return 1
    print(f"OK: {len(reservas)} reservas analizadas, {len(avisos)} descartadas -> {args.salida}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

- Los docstrings están abreviados en este informe; en vuestro `kpis.py` se quedan los de la plantilla.
- Las dos últimas líneas **no se tocan**: son las que arrancan `main` al escribir `python kpis.py …`.

7 · ¿Lo habéis visto todo?

- [ ] Sé **de dónde sale cada cifra** de `salida.json` (qué función la calcula).
- [ ] Puedo explicar con mis palabras **qué hace `main`** y en qué orden llama a las demás.
- [ ] He visto **fallar** las pruebas al principio (`1 de 12`) y **pasar** al final (`12 de 12`).
- [ ] He provocado **los tres errores** y sé qué mensaje y qué código sale en cada uno.
- [ ] Puedo explicar por qué una **fila mala se descarta con aviso** y un **fichero que falta para el programa**.
- [ ] Sé por qué la ocupación es un **proxy** y qué no se puede concluir de ella.
- [ ] He anotado **una decisión de diseño** para la review del jueves (por ejemplo, cómo definisteis la ocupación).